

Developer Guide

Vol. 7 -- Manual Dependency Injection

Topics covered in this volume:

- What is Dependency Injection and why it matters
 - Manual DI vs Hilt: trade-offs and when to use each
 - AppComponent -- the service locator pattern
 - CompositionLocal -- passing dependencies through the Compose tree
 - viewModelFactory and the initializer DSL
 - Full migration: Hilt to manual DI in this project
 - Dart / Flutter equivalent patterns
-

1. What is Dependency Injection?

Dependency Injection (DI) is a pattern where a class receives its dependencies from the outside instead of creating them itself. This makes code easier to test, swap, and reason about.

Without DI, a ViewModel creates its own repository:

```
class CartViewModel : ViewModel() {  
    // Tightly coupled -- CartViewModel OWNS CartRepositoryImpl.  
    // Cannot be tested without a real repository.  
    private val repo = CartRepositoryImpl()  
}
```

With DI, the repository is passed in (injected):

```
class CartViewModel(  
    private val repo: ICartRepository,    // <- injected from outside  
) : ViewModel()
```

Now CartViewModel works with ANY ICartRepository -- the real one, a fake one, a test double. The caller decides which implementation to provide.

Dart equivalent: passing a repository into a Controller constructor instead of calling ServiceLocator.instance inside the Controller.

2. Three DI Approaches in Android

2a. Manual DI (what this project uses)

You wire dependencies yourself -- no framework, no code generation. A central `AppContainer` holds all singletons. `ViewModels` receive their dependencies through a factory function.

Pros: transparent, fast builds, easy to debug, no annotation processing.

Cons: more boilerplate as the project grows; you manage lifetimes manually.

2b. Hilt (what this project used before)

Hilt is Google's DI framework for Android built on top of Dagger. You annotate classes with `@HiltViewModel`, `@Inject`, `@Singleton`, etc., and Hilt generates the wiring code at compile time.

Pros: less boilerplate for large projects; lifecycle-aware scoping handled automatically.

Cons: slow incremental builds due to annotation processing; harder to debug when something breaks; `@EntryPoint` workarounds needed for `Composables`.

2c. Service Locator

A global registry (e.g. `GetIt` in Dart/Flutter) where any class can ask for a dependency by type. Simpler than full DI but creates hidden dependencies -- classes reach into a global object instead of declaring their needs explicitly.

This project uses a hybrid: `AppContainer` is a service locator accessed via `CompositionLocal` -- dependencies are declared explicitly in each `ViewModel` factory call, not fetched from a global.

3. AppContainer -- The DI Root

`AppContainer` is a plain Kotlin class that constructs and holds all singleton dependencies. It is the single source of truth for which implementations are used.

```
// core/AppContainer.kt
class AppContainer(context: Context) {

    // Singleton instances -- created ONCE and reused for the app's lifetime.
    val sharedPreferencesManager = SharedPreferencesManager(context)
    val cartRepository: ICartRepository = CartRepositoryImpl()
    val favoritesRepository: IFavoritesRepository = FavoritesRepositoryImpl()
    val productRepository: IProductRepository = ProductRepositoryImpl()
    val orderHistoryRepository: IOrderHistoryRepository =
        OrderHistoryImpl(sharedPreferencesManager)
    val orderRepository: IOrderRepository =
        OrderRepositoryImpl(sharedPreferencesManager)
}
```

Key points:

- All properties are `val` -- they are created once when `AppContainer` is constructed.
- Interfaces are used for the type (`ICartRepository`, not `CartRepositoryImpl`) -- you can swap the implementation by changing one line here.
- `OrderRepositoryImpl` receives `SharedPreferencesManager` via its constructor -- no `@Inject`, no Hilt, just a plain constructor call.

Dart equivalent: a top-level `setup()` function that calls `GetIt.instance.registerSingleton(CartRepositoryImpl())`

4. App.kt -- Application Class

The Application class is created once when the process starts -- before any Activity or ViewModel. It is the right place to build AppContainer.

```
// App.kt
class App : Application() {
    lateinit var container: AppContainer

    override fun onCreate() {
        super.onCreate()
        container = AppContainer(this) // 'this' is the Application Context
    }
}
```

Before (with Hilt):

```
@HiltAndroidApp          // triggers Hilt code generation
class App : Application()
```

After (manual DI): no annotations -- just a plain Application subclass that builds the container. The AndroidManifest still points to this class via android:name=".App".

Dart equivalent: the main() function where you initialize GetIt or call WidgetsFlutterBinding.ensureInitialized() before runApp().

5. CompositionLocal -- Passing Dependencies Through Compose

Composable functions cannot use @Inject. To avoid passing AppContainer as a parameter through every composable in the tree, we use CompositionLocal -- Compose's built-in mechanism for implicit dependency passing.

5a. Defining the CompositionLocal

```
// core/LocalAppContainer.kt
val LocalAppContainer = staticCompositionLocalOf<AppContainer> {
    error("No AppContainer provided") // crash early if forgotten
}
```

staticCompositionLocalOf is used (not compositionLocalOf) because AppContainer never changes after the app starts -- no recomposition needed when it is read.

5b. Providing it in MainActivity

```
// MainActivity.kt
class MainActivity : ComponentActivity() {
    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        enableEdgeToEdge()
        val container = (application as App).container
        setContent {
            OnlineStoreClaudeTheme {
                CompositionLocalProvider(LocalAppContainer provides container) {
                    AppNavigation()
                }
            }
        }
    }
}
```

```

    }
  }
}

```

5c. Reading it in any Screen

```

@Composable
fun CartScreen(onBackClick: () -> Unit, onCheckoutClick: () -> Unit) {
    val container = LocalAppContainer.current // reads from the Compose tree
    val viewModel: CartViewModel =
        viewModel(factory = CartViewModel.factory(container.cartRepository))
    // ...
}

```

Dart equivalent: using `Provider.of(context)` or `context.read()` after wrapping the widget tree in a `Provider(create: { } => container, child: ...)`

6. viewModelFactory and the initializer DSL

Every ViewModel needs a Factory so the Jetpack ViewModel system can construct it with the right dependencies. We use the viewModelFactory DSL from `androidx.lifecycle.viewmodel`.

6a. Pattern -- companion object factory

```

class CartViewModel(
    private val cartRepository: ICartRepository,
) : ViewModel() {

    companion object {
        fun factory(cartRepository: ICartRepository) = viewModelFactory {
            initializer { CartViewModel(cartRepository) }
        }
    }
    // ... state, load(), removeItem(), etc.
}

```

The factory is a companion object function so the screen can call `CartViewModel.factory(repo)` without an instance.

6b. Using the factory in a screen

```

val container = LocalAppContainer.current
val viewModel: CartViewModel =
    viewModel(factory = CartViewModel.factory(container.cartRepository))

```

`viewModel()` is scoped to the current `NavBackStackEntry` inside a `NavHost`, so each destination gets its own ViewModel instance -- same behavior as `hiltViewModel()`.

6c. ViewModel with multiple dependencies

```

class ProductDetailViewModel(
    private val productId: Int,
    private val productRepository: IProductRepository,
    private val cartRepository: ICartRepository,

```

```

        private val favoritesRepository: IFavoritesRepository,
    ) : ViewModel() {

        companion object {
            fun factory(
                productId: Int,
                productRepository: IProductRepository,
                cartRepository: ICartRepository,
                favoritesRepository: IFavoritesRepository,
            ) = viewModelFactory {
                initializer {
                    ProductDetailViewModel(
                        productId, productRepository,
                        cartRepository, favoritesRepository,
                    )
                }
            }
        }
    }
}

```

Dart equivalent: passing the controller's dependencies in the `ChangeNotifierProvider.create()` callback:
`ChangeNotifierProvider(create: (_) => ProductDetailController( productId: id, productRepo: repo, cartRepo: cart))`

7. Full Migration: Hilt to Manual DI

The table below shows every change made to remove Hilt from this project.

File / Area	Before (Hilt)	After (Manual DI)
App.kt	@HiltAndroidApp class App : Application()	class App : Application() + lateinit var container: AppContainer + override fun onCreate() { container = AppContainer(this) }
MainActivity.kt	@AndroidEntryPoint class MainActivity	class MainActivity + CompositionLocalProvider(LocalAppContainer provides container) { AppNavigation() }
AppContainer.kt	Did not exist. Hilt's SingletonComponent held all singletons.	New file: plain class holding all singleton repository instances.
LocalAppContainer.kt	Did not exist. Composables used @EntryPoint + EntryPointAccessors.	New file: staticCompositionLocalOf<AppContainer>
AppModule.kt AppEntryPoint.kt	@Module @InstallIn(...) abstract class AppModule + @EntryPoint interface	DELETED -- no longer needed.
ViewModels	@HiltViewModel class XxxViewModel @Inject constructor(repo)	class XxxViewModel(repo) + companion object { fun factory(repo) = viewModelFactory { initializer { XxxViewModel(repo) } } }
OrderViewModel (was @AssistedInject)	@HiltViewModel(assistedFactory=...) + @AssistedInject constructor + @AssistedFactory interface	class OrderViewModel(orderRepo, cartRepo) + companion factory. All AssistedInject code removed.

Screens	hiltViewModel() OR hiltViewModel<VM, Factory>{ factory.create(...) }	val container = LocalAppContainer.current viewModel(factory = XxxViewModel.factory(container.xxxRepository))
Repositories	@Inject constructor() @Singleton	Plain constructor. Singleton enforced by AppContainer holding one instance.
SharedPreferencesManager	@Singleton @Inject constructor(@ApplicationContext context)	Plain class: SharedPreferencesManager(context) created in AppContainer.
build.gradle.kts	hilt.android plugin ksp plugin hilt.android dep hilt.compiler ksp dep hilt.navigation.compose dep	All Hilt and KSP entries removed.

8. Why Manual DI is Better for This Project

8a. Transparency

With Hilt, wiring happens invisibly in generated code. You annotate a class and trust that the right object appears. When it breaks, the error messages point to generated files.

With manual DI, the full wiring is in AppContainer.kt and each screen's viewModel() call. You can read exactly where every dependency comes from.

8b. No Annotation Processing

Hilt uses KSP (Kotlin Symbol Processing) to generate code at compile time. This adds several seconds to every incremental build. Manual DI has zero annotation processing -- the Kotlin compiler processes only your code.

8c. No @EntryPoint Workarounds

The previous codebase needed AppEntryPoint + EntryPointAccessors.fromApplication() in two screens just to pass SharedPreferencesManager into a manually-created repository. This was necessary because @AssistedInject forced manual construction but Composables cannot use @Inject.

With manual DI, every dependency flows naturally through AppContainer -- no special entry points needed.

8d. Easier Testing

```
// In a test, you can create a CartViewModel with a fake repository:
val fakeRepo = FakeCartRepository()
val viewModel = CartViewModel(fakeRepo)

// With Hilt you would need to create a test module and replace the binding.
// With manual DI -- just pass the fake directly.
```

8e. When to use Hilt instead

Hilt pays off in large multi-module projects where dozens of ViewModels share deeply nested dependency graphs. It handles scoping (Activity, Fragment, ViewModel scope) automatically and reduces the

boilerplate of building factories for every ViewModel. For a project of this size, manual DI is cleaner.

9. staticCompositionLocalOf vs compositionLocalOf

Compose has two ways to create a CompositionLocal:

compositionLocalOf

```
val LocalTheme = compositionLocalOf { LightTheme }  
// Reading compositionLocalOf invalidates ONLY the composables  
// that directly read it when the value changes.
```

staticCompositionLocalOf

```
val LocalAppContainer = staticCompositionLocalOf<AppContainer> {  
    error("No AppContainer provided")  
}  
// Reading staticCompositionLocalOf invalidates the ENTIRE subtree  
// when the value changes -- more expensive.  
// Use this when the value NEVER changes (like AppContainer).
```

AppContainer is constructed once in Application.onCreate() and never replaced, so staticCompositionLocalOf is the correct choice -- it avoids unnecessary recomposition tracking overhead.

Dart equivalent: using InheritedWidget vs InheritedNotifier. InheritedWidget (static) rebuilds descendants on any change. InheritedNotifier (dynamic) only rebuilds listeners.

10. Dependency Flow Diagram

```
Application.onCreate()  
|  
+-- AppContainer(context)  
    |  
    +-- SharedPreferencesManager(context)    [singleton]  
    +-- CartRepositoryImpl()                  [singleton]  
    +-- FavoritesRepositoryImpl()              [singleton]  
    +-- ProductRepositoryImpl()                [singleton]  
    +-- OrderHistoryImpl(sharedPrefs)          [singleton]  
    +-- OrderRepositoryImpl(sharedPrefs)       [singleton]  
  
MainActivity.setContent {  
    CompositionLocalProvider(LocalAppContainer provides container) {  
        AppNavigation() {  
            CartScreen() {  
                val container = LocalAppContainer.current  
                viewModel(factory = CartViewModel.factory(container.cartRepository))  
                // ^  
                // singleton from AppContainer, not a new instance  
            }  
        }  
    }  
}
```

Every screen reads from LocalAppContainer and passes the appropriate singleton repository to its ViewModel factory. No repository is ever constructed more than once.

End of Vol. 7